

Advanced Optimization

Chapter 5 — Conjugate Gradient Methods

Hongseok Kim

Department of Electronic Engineering, Sogang University

Fall 2026

Reading: Nocedal & Wright, *Numerical Optimization*, 2nd ed., Chapter 5

Two uses of conjugate gradients

Linear solver (§5.1)

Hestenes–Stiefel, 1950s.

Solves $Ax = b$ with $A \succ 0$ using only products Ap . No factorization.

The convergence rate is determined by the **eigenvalues of A** .

Optimizer (§5.2)

Fletcher–Reeves, 1960s.

Minimizes a general f using only ∇f and a few vectors. **No matrix is stored.**

Faster than steepest descent.

Linear CG is used again in Newton–CG and CG–Steihaug (Ch. 7), projected CG (Ch. 16), and interior-point methods (Ch. 19).

A linear system and a quadratic minimization problem

$$Ax = b \quad (5.1) \quad \Longleftrightarrow \quad \min_x \phi(x) = \frac{1}{2}x^T Ax - b^T x \quad (5.2)$$

A is symmetric positive definite. Both problems have the same unique solution x^* .

The gradient of ϕ equals the **residual** of the linear system:

$$\nabla \phi(x) = Ax - b =: r(x), \quad r_k = Ax_k - b. \quad (5.3-5.4)$$

How to read §5.1

A statement about the residual r_k is a statement about the gradient $\nabla \phi(x_k)$.

“Minimize ϕ along p_k ” is an exact line search on a quadratic.

Two formulas used throughout §5.1

Exact step length. Along $x_k + \alpha p_k$,

$$\begin{aligned}\frac{d}{d\alpha}\phi(x_k + \alpha p_k) &= p_k^T (A(x_k + \alpha p_k) - b) = r_k^T p_k + \alpha p_k^T A p_k = 0 \\ \implies \alpha_k &= -\frac{r_k^T p_k}{p_k^T A p_k}.\end{aligned}\tag{5.7}$$

Residual update. From $x_{k+1} = x_k + \alpha_k p_k$ and $r = Ax - b$,

$$r_{k+1} = r_k + \alpha_k A p_k.\tag{5.10}$$

With the exact α_k : $r_{k+1}^T p_k = 0$. The new gradient is orthogonal to the direction just used.

Conjugate directions

Definition

Nonzero vectors $\{p_0, \dots, p_l\}$ are **conjugate** with respect to $A \succ 0$ if

$$p_i^T A p_j = 0 \quad \text{for all } i \neq j. \quad (5.5)$$

- ▶ Conjugacy is orthogonality in the inner product $\langle u, v \rangle_A = u^T A v$.
- ▶ A conjugate set is linearly independent (homework). Hence n conjugate vectors form a basis of $\mathbb{R}^n$.
- ▶ Example: the eigenvectors of A are orthogonal and also conjugate.

The conjugate direction method

Method

Given x_0 and a conjugate set $\{p_0, \dots, p_{n-1}\}$:

$$x_{k+1} = x_k + \alpha_k p_k, \quad \alpha_k = -\frac{r_k^T p_k}{p_k^T A p_k}. \quad (5.6-5.7)$$

Theorem 5.1

For any $x_0 \in \mathbb{R}^n$, the sequence $\{x_k\}$ generated by the conjugate direction method converges to x^* in at most n steps.

Here the directions are given in advance. CG generates them during the iteration.

Proof of Theorem 5.1

The p_i form a basis of $\mathbb{R}^n$, so $x^* - x_0 = \sigma_0 p_0 + \sigma_1 p_1 + \cdots + \sigma_{n-1} p_{n-1}$.

Multiply by $p_k^T A$. By conjugacy all terms except the k -th vanish:

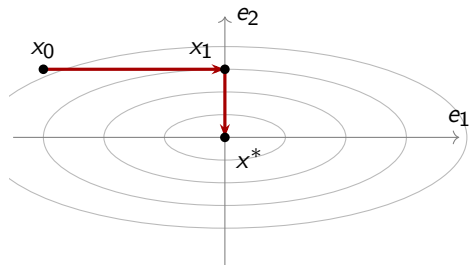
$$\sigma_k = \frac{p_k^T A(x^* - x_0)}{p_k^T A p_k}. \quad (5.8)$$

The iterates satisfy $x_k = x_0 + \alpha_0 p_0 + \cdots + \alpha_{k-1} p_{k-1}$, so $p_k^T A(x_k - x_0) = 0$ and

$$p_k^T A(x^* - x_0) = p_k^T A(x^* - x_k) = p_k^T (b - Ax_k) = -p_k^T r_k.$$

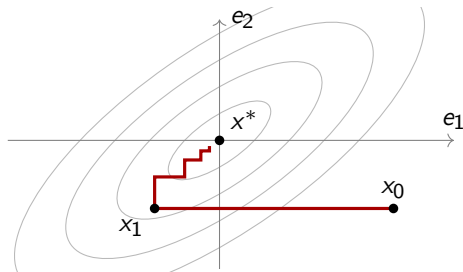
Comparing with (5.7): $\sigma_k = \alpha_k$ for every k . Hence $x_n = x^*$. $\square$

Geometry (Figures 5.1 and 5.2)



A diagonal: successive minimization along e_1, e_2 reaches x^* in n steps.

With $S = [p_0 \cdots p_{n-1}]$ and $\hat{x} = S^{-1}x$ (5.9), $S^T A S$ is diagonal: in the variables $\hat{x}$ the right picture becomes the left one.



A not diagonal: the same procedure does not terminate in n steps.

Theorem 5.2: expanding subspace minimization

Theorem 5.2

Let $x_0 \in \mathbb{R}^n$ be any starting point and let $\{x_k\}$ be generated by the conjugate direction method. Then

$$r_k^T p_i = 0 \quad \text{for } i = 0, 1, \dots, k-1, \quad (5.11)$$

and x_k is the minimizer of ϕ over the set

$$\{x \mid x = x_0 + \text{span}\{p_0, p_1, \dots, p_{k-1}\}\}. \quad (5.12)$$

Theorem 5.1: exact solution after n steps.

Theorem 5.2: after k steps, x_k is optimal over a k -dimensional affine set.

Read: proof of Theorem 5.2, pp. 106–107.

Consequences of Theorem 5.2

- ▶ $\phi(x_k)$ is nonincreasing, and x_k is the best point in $x_0 + \text{span}\{p_0, \dots, p_{k-1}\}$. Stopping at any $k < n$ returns a usable approximation. (This is what CG–Steihaug and Newton–CG rely on.)
- ▶ Any conjugate set can be used:
 - eigenvectors of A — requires an eigendecomposition;
 - Gram–Schmidt conjugation of a basis — requires storing all previous directions.Both are too expensive for large n .
- ▶ The conjugate gradient method computes p_k from r_k and p_{k-1} only.

Conjugate gradient: the search direction

Each direction is a combination of the steepest descent direction and the *previous direction only*:

$$p_k = -r_k + \beta_k p_{k-1}, \quad p_0 = -r_0. \quad (5.13)$$

β_k is determined by the requirement $p_{k-1}^T A p_k = 0$:

$$\beta_k = \frac{r_k^T A p_{k-1}}{p_{k-1}^T A p_{k-1}}.$$

What is and is not imposed

Conjugacy is imposed only between p_k and p_{k-1} . Conjugacy with $p_0, \dots, p_{k-2}$ is not imposed; it holds anyway (Theorem 5.3). This is why no earlier direction needs to be stored.

Theorem 5.3

Krylov subspace of degree k for r_0 : $\mathcal{K}(r_0; k) = \text{span}\{r_0, Ar_0, \dots, A^k r_0\}$ (5.15).

Theorem 5.3

Suppose the k -th iterate of the CG method is not the solution x^* . Then

$$r_k^T r_i = 0, \quad i = 0, \dots, k-1, \quad (5.16)$$

$$\text{span}\{r_0, r_1, \dots, r_k\} = \mathcal{K}(r_0; k), \quad (5.17)$$

$$\text{span}\{p_0, p_1, \dots, p_k\} = \mathcal{K}(r_0; k), \quad (5.18)$$

$$p_k^T A p_i = 0, \quad i = 0, \dots, k-1. \quad (5.19)$$

Therefore $\{x_k\}$ converges to x^* in at most n steps.

Reading Theorem 5.3

- ▶ (5.16): the residuals (gradients) are mutually *orthogonal*.
(5.19): the search directions are mutually *conjugate*.
The name “conjugate gradient” is inaccurate: the gradients are not conjugate.
- ▶ (5.17)–(5.18): $x_{k+1} \in x_0 + \mathcal{K}(r_0; k)$. CG is a **Krylov subspace method**.
- ▶ Combined with Theorem 5.2: x_{k+1} minimizes ϕ over $x_0 + \mathcal{K}(r_0; k)$.
- ▶ The proof uses $p_0 = -r_0$. For any other p_0 the theorem is false. This matters for restarts in §5.2.

Read: proof of Theorem 5.3 (induction on k), pp. 109–111.

Algorithm 5.2 (CG)

Given x_0 ; $r_0 \leftarrow Ax_0 - b$, $p_0 \leftarrow -r_0$, $k \leftarrow 0$

while $r_k \neq 0$

$$\alpha_k \leftarrow r_k^T r_k / p_k^T A p_k \quad (5.24a)$$

$$x_{k+1} \leftarrow x_k + \alpha_k p_k \quad (5.24b)$$

$$r_{k+1} \leftarrow r_k + \alpha_k A p_k \quad (5.24c)$$

$$\beta_{k+1} \leftarrow r_{k+1}^T r_{k+1} / r_k^T r_k \quad (5.24d)$$

$$p_{k+1} \leftarrow -r_{k+1} + \beta_{k+1} p_k \quad (5.24e)$$

$$k \leftarrow k + 1$$

This is (5.13) with the exact step length, simplified using (5.10) and (5.11).

Read: derivation, p. 111.

Cost, storage, and floating-point arithmetic

- ▶ Per iteration: **one product** Ap_k , two inner products $(p_k^T Ap_k, r_{k+1}^T r_{k+1})$, three vector updates. Storage: x, r, p, Ap .
- ▶ A enters only through $p \mapsto Ap$. It is never factored or modified. A routine that returns Ap is sufficient.
- ▶ Small n : use Gaussian elimination or another factorization; these are less sensitive to rounding errors. CG is for large n .
- ▶ (Not in the book.) In floating point, (5.16) and (5.19) degrade as k grows, and exact termination in n steps is not observed. In practice CG is run as an iterative method and stopped when $\|r_k\| \leq \text{tol}$.

Link to trust-region methods: CG–Steihaug (Algorithm 7.2)

Trust-region subproblem: $\min_p m(p) = g^T p + \frac{1}{2} p^T B p$ s.t. $\|p\| \leq \Delta$.

Run Algorithm 5.2 on $Bp = -g$ from $z_0 = 0$, with directions d_j and iterates z_j .

- ▶ $d_0 = -g$: the first CG step, truncated to the region if necessary, is the Cauchy point.
- ▶ Theorem 5.2: $m(z_j)$ decreases with j . Every later step is at least as good as the Cauchy point.
- ▶ $\|z_j\|$ increases strictly with j (Theorem 7.3), because $d_i^T d_j > 0$. So the iteration can stop at the first crossing of the boundary.
- ▶ If $d_j^T B d_j \leq 0$: move along d_j to the boundary and stop.

Read: Algorithm 7.2 and Theorem 7.3, pp. 171–173.

Linear CG so far

1. With conjugate directions the quadratic separates: n exact one-dimensional minimizations give x^* (Theorem 5.1).
2. After k steps, x_k is optimal over $x_0 + \text{span}\{p_0, \dots, p_{k-1}\}$ (Theorem 5.2).
3. CG: $p_k = -r_k + \beta_k p_{k-1}$ with $p_0 = -r_0$ gives conjugate directions, orthogonal residuals, and the span $\mathcal{K}(r_0; k)$ (Theorem 5.3).
4. Cost: one product Ap per iteration, four vectors of storage.

Next. How many iterations are needed? The answer depends on the eigenvalues of A . Then preconditioning and nonlinear CG.

Rate of convergence: the polynomial view

By (5.18), $x_{k+1} = x_0 + P_k^*(A) r_0$ for a polynomial P_k^* of degree k (5.26).

Error measure: $\|z\|_A^2 = z^T A z$, for which $\frac{1}{2} \|x - x^*\|_A^2 = \phi(x) - \phi(x^*)$ (5.28).

By Theorem 5.2, x_{k+1} minimizes ϕ over $x_0 + \mathcal{K}(r_0; k)$. Hence P_k^* solves

$$\min_{P_k} \|x_0 + P_k(A) r_0 - x^*\|_A. \quad (5.29)$$

Optimality

No method whose $(k+1)$ -th iterate lies in $x_0 + \mathcal{K}(r_0; k)$ has a smaller A -norm error than CG.

The rate depends on the eigenvalues of A

Let $\lambda_1 \leq \dots \leq \lambda_n$ be the eigenvalues of A . Expand $x_0 - x^*$ in the eigenvectors of A and use $r_0 = A(x_0 - x^*)$ in (5.29):

$$\|x_{k+1} - x^*\|_A^2 \leq \min_{P_k} \max_{1 \leq i \leq n} [1 + \lambda_i P_k(\lambda_i)]^2 \|x_0 - x^*\|_A^2. \quad (5.33)$$

The rate theory of CG

$Q_{k+1}(\lambda) = 1 + \lambda P_k(\lambda)$ is any polynomial of degree $k + 1$ with $Q_{k+1}(0) = 1$. CG converges as fast as such a polynomial can be made small *on the eigenvalues of A* . The rate depends on the location of the eigenvalues, not on n .

To obtain a bound it is enough to exhibit one polynomial.

Read: from (5.29) to (5.33), pp. 113–114.

Theorem 5.4: r distinct eigenvalues

Theorem 5.4

If A has only r distinct eigenvalues, then the CG iteration terminates at the solution in at most r iterations.

Proof. Let the eigenvalues take the values $\tau_1 < \dots < \tau_r$ and define

$$Q_r(\lambda) = \prod_{j=1}^r \left(1 - \frac{\lambda}{\tau_j}\right).$$

Then $Q_r(\lambda_i) = 0$ for every i and $Q_r(0) = 1$. Since $Q_r(\lambda) - 1$ has a root at $\lambda = 0$, $\bar{P}_{r-1}(\lambda) = (Q_r(\lambda) - 1)/\lambda$ is a polynomial of degree $r - 1$. Put $k = r - 1$ in (5.33): $\|x_r - x^*\|_A = 0$. $\square$

Theorem 5.5: bound from the eigenvalue distribution

Theorem 5.5

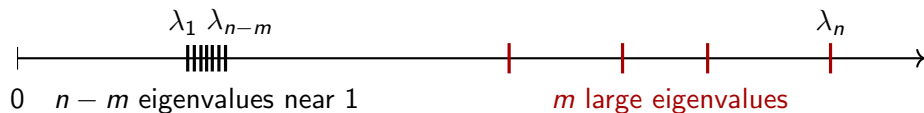
If A has eigenvalues $\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$, then

$$\|x_{k+1} - x^*\|_A^2 \leq \left(\frac{\lambda_{n-k} - \lambda_1}{\lambda_{n-k} + \lambda_1} \right)^2 \|x_0 - x^*\|_A^2. \quad (5.35)$$

Idea (the book gives no details; proof in Luenberger). Choose Q_{k+1} with roots at the k largest eigenvalues $\lambda_n, \dots, \lambda_{n-k+1}$ and at the midpoint of λ_1 and λ_{n-k} . Its maximum over the remaining eigenvalues is the factor above.

$k = 0$ gives $(\lambda_n - \lambda_1)/(\lambda_n + \lambda_1)$, the steepest descent factor. After $k + 1$ iterations the bound involves only the $n - k$ smallest eigenvalues.

Clustered eigenvalues (Figures 5.3–5.5)



Let $\epsilon = \lambda_{n-m} - \lambda_1$. Theorem 5.5 with $k = m$:

$$\|x_{m+1} - x^*\|_A \approx \epsilon \|x_0 - x^*\|_A.$$

In general: r clusters of eigenvalues $\Rightarrow$ a good estimate of x^* in about r iterations.

Figure 5.4: five large eigenvalues, the rest in $[0.95, 1.05]$. The sharp decrease predicted at iteration 6 occurs one iteration earlier (upper bound). Figure 5.5: four clusters.

The condition-number bound

With $\kappa(A) = \|A\|_2 \|A^{-1}\|_2 = \lambda_n / \lambda_1$:

$$\|x_k - x^*\|_A \leq 2 \left(\frac{\sqrt{\kappa(A)} - 1}{\sqrt{\kappa(A)} + 1} \right)^k \|x_0 - x^*\|_A. \quad (5.36)$$

- ▶ Uses only the two extreme eigenvalues. It is often a large overestimate; the clustering of the eigenvalues determines the actual behavior.
- ▶ Steepest descent, (3.29): the same form with κ in place of $\sqrt{\kappa}$.
- ▶ (5.36) is obtained with Chebyshev polynomials. The book does not derive it.

CG and steepest descent compared

Factor per iteration: SD $(\kappa - 1)/(\kappa + 1)$, CG $(\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$.
Iterations needed to reduce the A -norm error by a factor 10^6 :

κ	SD factor	CG factor	SD iterations	CG iterations
10^2	0.980	0.818	≈ 700	≈ 70
10^4	0.9998	0.980	$\approx 70,000$	≈ 700
10^6	0.999998	0.998	$\approx 7,000,000$	$\approx 7,000$

- ▶ CG with $\kappa = 10^4$ needs as many iterations as SD with $\kappa = 10^2$.
- ▶ A large κ still slows CG down. The remedy is to change the eigenvalue distribution of the system: **preconditioning**.

Preconditioning

Change variables $\hat{x} = Cx$ (5.37), with C nonsingular. The quadratic becomes

$$\hat{\phi}(\hat{x}) = \frac{1}{2} \hat{x}^T (C^{-T} A C^{-1}) \hat{x} - (C^{-T} b)^T \hat{x}. \quad (5.38)$$

CG applied to $\hat{\phi}$ converges according to the eigenvalues of $C^{-T} A C^{-1}$, not those of A .

Goal

Choose C such that $C^{-T} A C^{-1}$ has a *small condition number* or, better, *clustered eigenvalues*. The ideal is $C^{-T} A C^{-1} \approx I$.

The change of variables is never carried out. The resulting method uses only $M = C^T C \succ 0$, and only through solves $My = r$.

Algorithm 5.3 (preconditioned CG)

Given x_0 and M ; $r_0 \leftarrow Ax_0 - b$; solve $My_0 = r_0$; $p_0 \leftarrow -y_0$; $k \leftarrow 0$

while $r_k \neq 0$

$$\alpha_k \leftarrow r_k^T y_k / p_k^T A p_k \quad (5.39a)$$

$$x_{k+1} \leftarrow x_k + \alpha_k p_k \quad (5.39b)$$

$$r_{k+1} \leftarrow r_k + \alpha_k A p_k \quad (5.39c)$$

$$\text{solve } My_{k+1} = r_{k+1} \quad (5.39d)$$

$$\beta_{k+1} \leftarrow r_{k+1}^T y_{k+1} / r_k^T y_k \quad (5.39e)$$

$$p_{k+1} \leftarrow -y_{k+1} + \beta_{k+1} p_k; \quad k \leftarrow k + 1 \quad (5.39f)$$

Remarks on Algorithm 5.3

- ▶ $M = I$ gives Algorithm 5.2.
- ▶ Additional cost per iteration: **one solve $My = r$** .
- ▶ The orthogonality of the residuals becomes $r_i^T M^{-1} r_j = 0$ for all $i \neq j$ (5.40).
- ▶ M does not have to be formed. A routine that returns $M^{-1}r$ is sufficient (a factorization, a coarse-grid solve, a few sweeps of another iteration).
- ▶ The requirements conflict: M close to A (few iterations), but $My = r$ cheap to solve and M cheap to construct. No preconditioner is best for all problems.

Practical preconditioners

- ▶ **Problem-specific.** For a discretized PDE, define M from a coarser discretization of the same problem.
- ▶ **General-purpose.** SSOR, banded preconditioners, incomplete Cholesky.
- ▶ **Incomplete Cholesky.** $A \approx \tilde{L}\tilde{L}^T$ with $\tilde{L}$ about as sparse as the lower triangle of A ; $M = \tilde{L}\tilde{L}^T$. Solving $My = r$ costs about one product Ap . The factorization can break down or lose positive definiteness.

Where CG applies in power-system problems

CG requires $A \succ 0$. The power-flow Jacobian is nonsymmetric and the interior-point KKT matrix is symmetric indefinite, so CG does not apply to them directly. It applies after a reduction to a positive definite system.

Fletcher–Reeves (Algorithm 5.4)

Two changes to Algorithm 5.2: (1) α_k from a line search on f ; (2) r_k replaced by ∇f_k .

Given x_0 ; evaluate $f_0, \nabla f_0$; $p_0 \leftarrow -\nabla f_0$, $k \leftarrow 0$

while $\nabla f_k \neq 0$

 compute α_k ; $x_{k+1} \leftarrow x_k + \alpha_k p_k$; evaluate ∇f_{k+1}

$$\beta_{k+1}^{\text{FR}} \leftarrow \frac{\nabla f_{k+1}^T \nabla f_{k+1}}{\nabla f_k^T \nabla f_k} \quad (5.41a)$$

$$p_{k+1} \leftarrow -\nabla f_{k+1} + \beta_{k+1}^{\text{FR}} p_k; \quad k \leftarrow k + 1 \quad (5.41b)$$

Each iteration needs only f and ∇f . No matrix, a few vectors of storage.

Choices of β and the line search

$$\beta_{k+1}^{\text{PR}} = \frac{\nabla f_{k+1}^T (\nabla f_{k+1} - \nabla f_k)}{\|\nabla f_k\|^2} \quad (5.44)$$

$$\beta_{k+1}^+ = \max\{\beta_{k+1}^{\text{PR}}, 0\} \quad (5.45)$$

- ▶ On a strongly convex quadratic with exact line search, $\nabla f_{k+1}^T \nabla f_k = 0$, so FR, PR and HS coincide. On general f they differ.
- ▶ Others: Hestenes–Stiefel (5.46), Dai–Yuan (5.49), Hager–Zhang (5.50). No single formula is best on all problems.
- ▶ **Line search.** With an inexact line search, p_k can be an **ascent direction** (5.42). Under strong Wolfe with $c_2 < \frac{1}{2}$, FR gives descent (Lemma 5.6).
- ▶ Typical: $c_2 = 0.1$ for CG; $c_2 = 0.9$ for Newton and quasi-Newton.

FR versus PR; restarts

Suppose p_k is nearly orthogonal to $-\nabla f_k$. Then the step is very small and $\nabla f_{k+1} \approx \nabla f_k$.

FR

$$\beta_{k+1}^{\text{FR}} \approx 1 \Rightarrow p_{k+1} \approx p_k$$

The poor direction is repeated.

PR (also PR+, HS)

$$\beta_{k+1}^{\text{PR}} \approx 0 \Rightarrow p_{k+1} \approx -\nabla f_{k+1}$$

A restart along steepest descent.

Example in the book ($n = 100$): FR thousands of iterations, PR 37.

- ▶ **Restart** (set $\beta_k = 0$) when consecutive gradients are far from orthogonal:
 $|\nabla f_k^T \nabla f_{k-1}| / \|\nabla f_k\|^2 \geq \nu, \quad \nu = 0.1 \text{ (5.52)}.$
- ▶ **Theory.** FR converges (Thm. 5.7). PR can fail (Thm. 5.8); PR+ does not.
Read: pp. 125–131 (optional).

Nonlinear CG in practice, and the link to L-BFGS

- ▶ **Strength.** Only f and ∇f ; a few vectors of storage; no matrix.
- ▶ **Weakness.** Sensitive to the choice of β and to the line-search accuracy.

Memoryless BFGS (§7.2)

Keep only the most recent pair (s_k, y_k) and reset the Hessian approximation to I at every iteration (7.22). With an exact line search this is the Hestenes–Stiefel CG method.

- ▶ L-BFGS keeps m pairs instead of one. It began as an attempt to improve nonlinear CG. **Read:** pp. 180–181.
- ▶ **Remarks (not in the book).** For large smooth problems L-BFGS is often preferred. With constant β and step length, $p_k = -\nabla f_k + \beta p_{k-1}$ is the momentum (heavy-ball) iteration of machine learning; conjugacy is lost.

Chapter 5 in one table

	Linear CG (§5.1)	Nonlinear CG (§5.2)
Problem	$Ax = b, A \succ 0$	$\min f$, general f
Direction	$p_k = -r_k + \beta_k p_{k-1}$	$p_k = -\nabla f_k + \beta_k p_{k-1}$
Step length	exact: $r_k^T r_k / p_k^T A p_k$	strong Wolfe, $c_2 < \frac{1}{2}$
β_k	$r_k^T r_k / r_{k-1}^T r_{k-1}$	FR, PR, PR+, HS, ...
Key property	conjugacy; optimal over $\mathcal{K}(r_0; k)$	descent (Lemma 5.6)
Termination	$\leq n$ steps; $\leq r$ for r distinct eigenvalues	$\liminf \ \nabla f_k\ = 0$ (FR)
Rate	$\sqrt{\kappa}$ in place of κ ; clustering	n -step quadratic (restarts)
Cost / iteration	one product Ap	$f, \nabla f$ in the line search
Acceleration	preconditioner $M \approx A$	restarts; choice of β

Three results to carry forward

1. **Conjugacy separates the problem.** In a conjugate basis an n -dimensional quadratic is n independent one-dimensional problems.
2. **The eigenvalue distribution determines the rate, not n .** Preconditioning changes the distribution.
3. **CG is optimal over the Krylov subspace (Theorems 5.2, 5.3).** For this reason CG can be stopped early: CG–Steihaug, Newton–CG.

Next: Chapter 6 — quasi-Newton methods.

Homework

1. Show that a conjugate set of vectors is linearly independent. (Exercise 5.2)
2. By hand, run Algorithm 5.2 on $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$, $b = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, $x_0 = 0$. Check that $r_1^T r_0 = 0$, $p_1^T A p_0 = 0$, and $x_2 = x^*$.
3. Show that $\beta^{\text{FR}} = \beta^{\text{PR}}$ on a strongly convex quadratic with exact line search. (Use (5.16).)
4. Implement Algorithm 5.2. Build $A \succ 0$ with $n = 100$ and exactly 5 distinct eigenvalues. Plot $\|r_k\|$ and confirm termination in 5 iterations (Theorem 5.4).
5. By hand, compute the Cholesky factor L of A (Algorithm A.2, p. 608). (a) Which entry of L is fill-in? (b) Set it to zero and recompute the rest (incomplete Cholesky, $\tilde{L}$). Which entry of $\tilde{L}\tilde{L}^T$ differs from A ?

$$A = \begin{bmatrix} 4 & 2 & 4 & 0 \\ 2 & 5 & 0 & 2 \\ 4 & 0 & 6 & 1 \\ 0 & 2 & 1 & 14 \end{bmatrix}$$